Rebranching integer programming

Jin-Guo Liu and *Zhong-Yi Ni*

Description

We use the optimal branching technique [1] to speed up the integer programming problem[2].

Integer programming is a searching problem. It searches exponentially large solution space for finding the one with the largest objective value. The basic idea of rebranch is to devide the solution space into multiple zones, each zone should have a smaller search space.

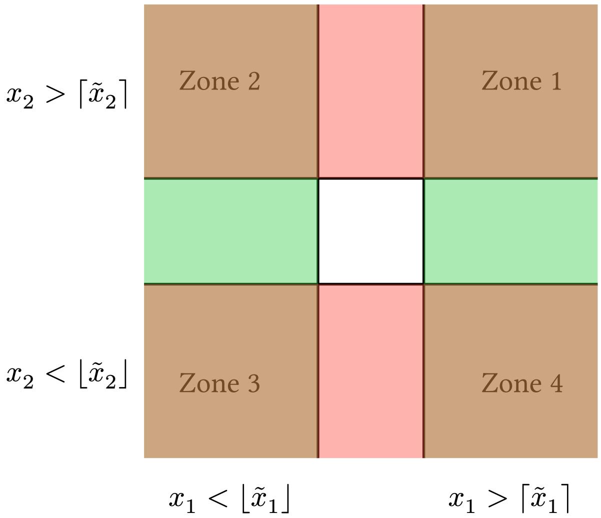
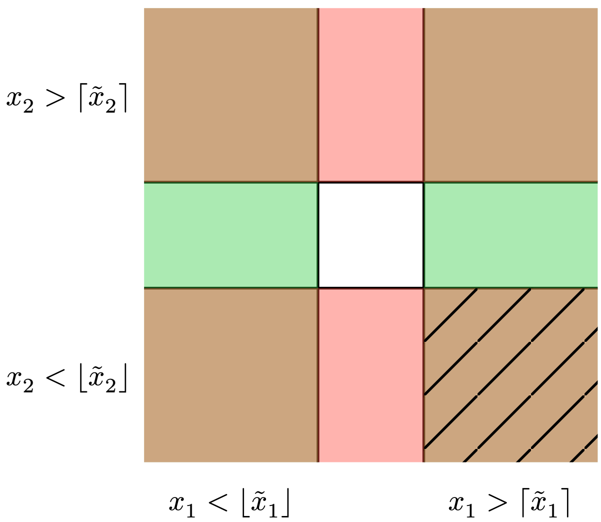


Figure 1: The branching on two variables divide the search space into 4 zones. Each zone corresponds to adding different constraints to the reduce the search space. Variables with "tilde" are solutions to the relaxed linear programming problem.

When branching over k variables, we can divide the search space into 2^k zones. Each zone has k extra constraints compared to the previous zone. These constraints reduces the search space by a certain factor. In our discussion, we assume each constraint halves the search space, and reduces the problem size by 1. So, the **measure** here is the number of constraints to be added to reach the optimal solution.

Through **bounding**, we can reduce the search space. For example, the following shaded zone is ruled out by the bounding.



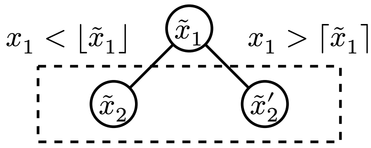
The **rebranching** is a technique to reduce the tree size by reorganizing the search tree. In this case, the rebranching produces two branches instead of three in the original tree:

$$\begin{cases} x_1 < \lfloor \tilde{x}_1 \rfloor \\ x_1 > \lceil \tilde{x}_1 \rceil \text{ and } x_2 > \lceil \tilde{x}_2 \rceil \end{cases}$$

The expected time complexity is determined by $1 = \gamma^{-2} + \gamma^{-1}$, which is $\gamma \approx 1.618$. This is smaller than the 1.73 given by the original branching: $1 = 3\gamma^{-2}$.

Deciding which subset of variables to branch

We introduce a **score function** for each variable, which is determined by some heuristic. First pick a variable x_1 with the highest score, which produces two branches.



x_2: the variable with the highest average score

On each branch, we evaluate the score of the remaining variables. x_2 is chosen to be the variable with the highest score averaged over the two branches.

Each branch is associated with a **upper bound** and **lower bound** of the objective value (to be maximized). If the upper bound of one branch is infeasible, or its upper bound is lower than the lower bound of another branch, we can prune the branch. The upper bound is computed with the linear programming relaxation, while the lower bound is a bit tricky. For simplicity, we can round the variables to the nearest integer, and compute the objective value.

Bibliography

[1] X.-Z. Gao, Y.-J. Wang, P. Zhang, and J.-G. Liu, “Automated Discovery of Branching Rules with Optimal Complexity for the Maximum Independent Set Problem,” no. arXiv:2412.07685. arXiv, Dec. 2024. doi: 10.48550/arXiv.2412.07685.

[2] T. Achterberg, “SCIP: Solving Constraint Integer Programs,” *Mathematical Programming Computation*, vol. 1, no. 1, pp. 1–41, Jul. 2009, doi: 10.1007/s12532-008-0001-1.